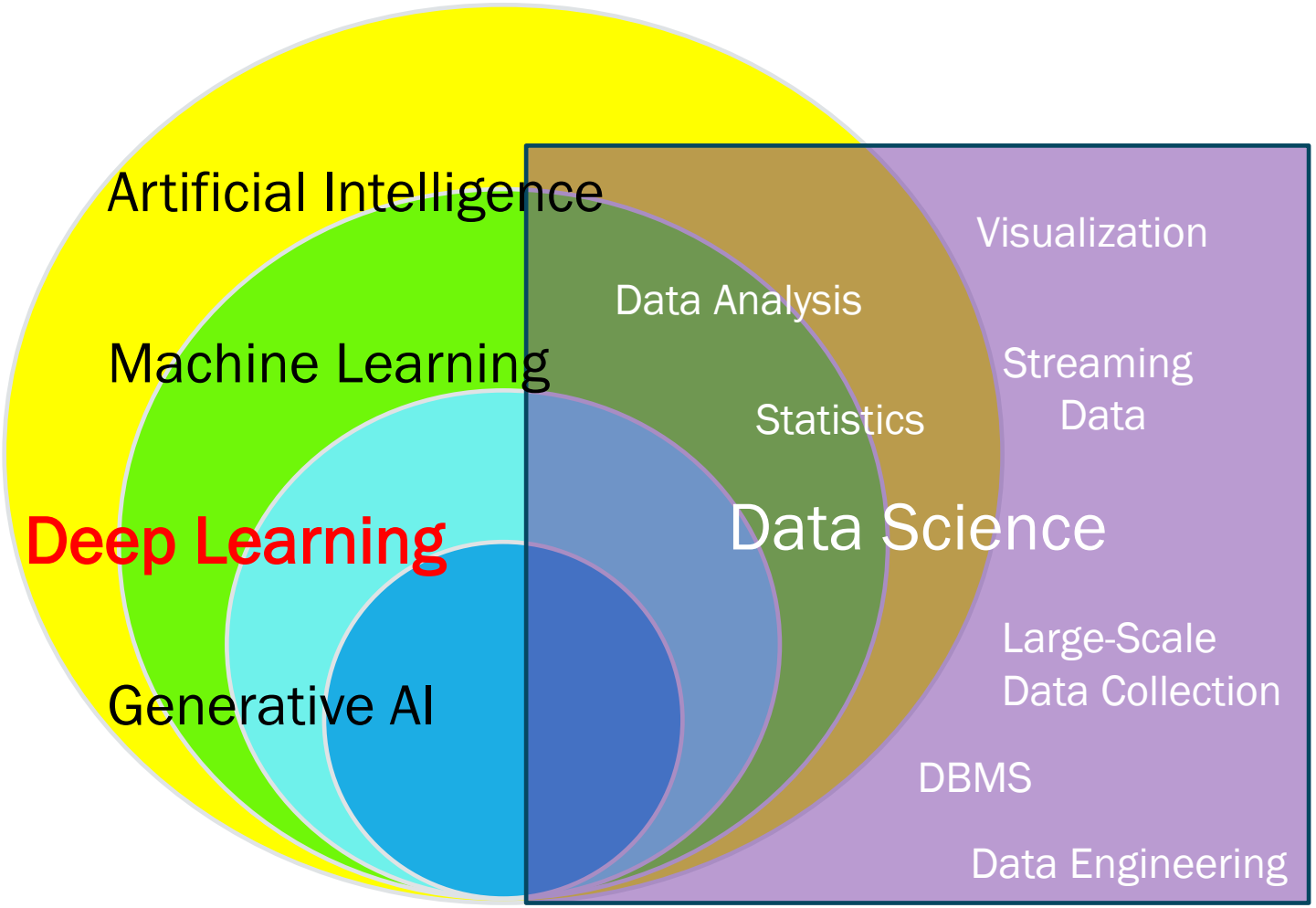
DEEP LEARNING NETWORKS

WHERE DOES DEEP LEARNING FIT?



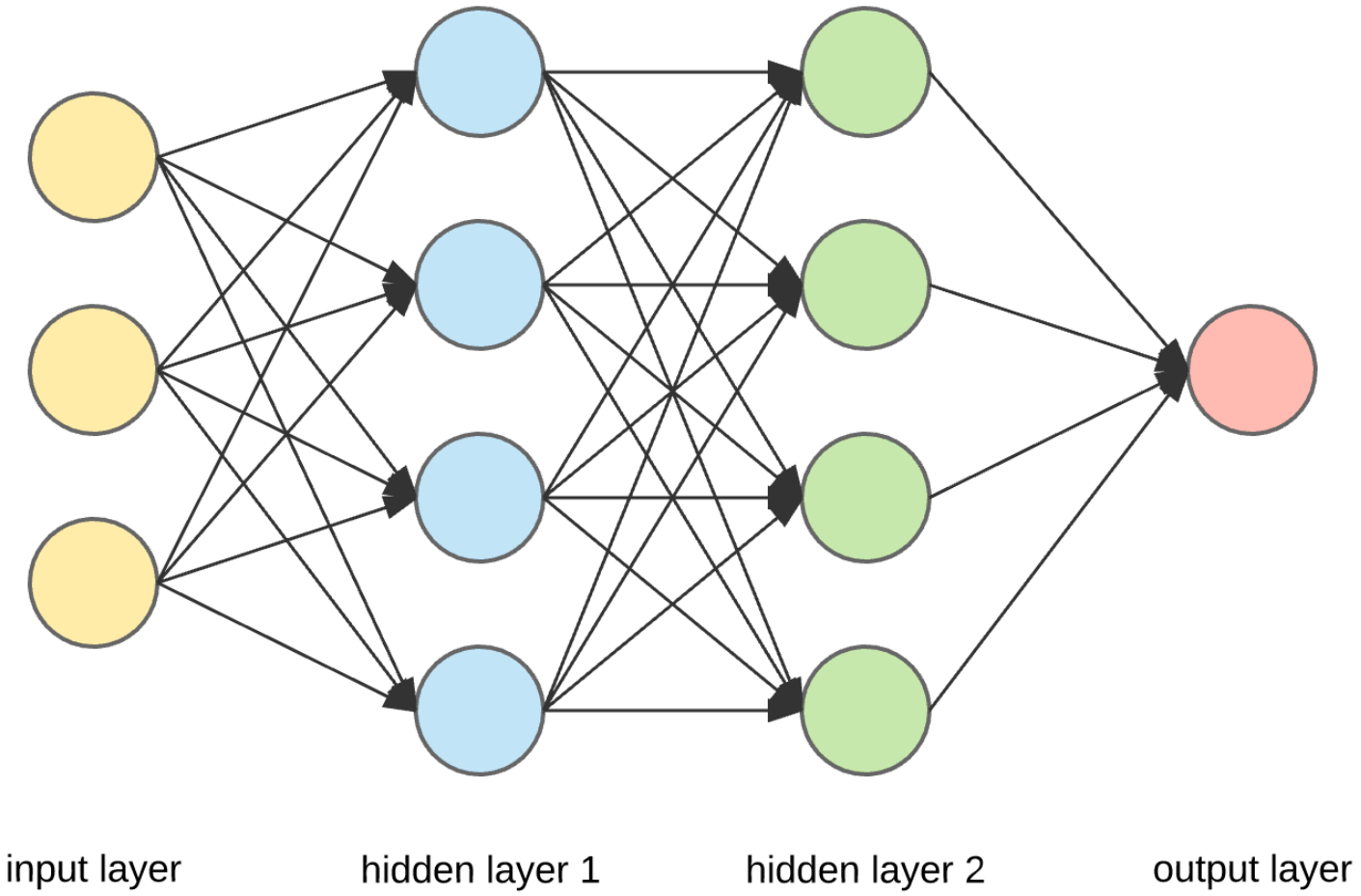
Deep Learning

- Deep learning is a specialized subset of machine learning that employs artificial neural networks with multiple layers to model and understand complex patterns in data.
- This approach enables systems to learn from vast amounts of unstructured data, such as images, audio, and text, by processing information through hierarchical layers that extract increasingly abstract features.
- Key Ideas: Convolutional Neural Networks, Recurrent NN, LSTM, GAN, Transformer Networks.

BYU

DEEP NEURAL NETWORK ARCHITECTURE

- DNNs are structured into layers:
  - **Input Layer**: Takes the features of the data.
  - **Hidden Layers**: Learn complex patterns. (Two or many more)
  - **Output Layer**: Produces the final predictions.
- Deep Neural Networks are technically defined as any network with more than one hidden layer.
- But in practice, they usually have many hidden layers.



BYU

DIFFERENT TYPES OF DEEP NEURAL NETWORKS

Model	Primary Use	Key Idea	Typical Applications
CNN	Image data	Convolutions extract spatial features	Image classification, detection
RNN	Sequential data	Cyclic connections for temporal patterns	Time series, speech
LSTM	Sequential data	Memory + gating for long dependencies	Translation, speech synthesis
Transformer	NLP, vision, speech	Self-attention, parallelizable	Language models, ViTs
U-Net	Medical/segmentation	Encoder–decoder + skip connections	Biomedical segmentation
GANs	Image/video generation	Generator vs. discriminator	Image synthesis, style transfer
Vision Transformer (ViT)	Vision tasks	Patch embeddings + transformer blocks	Image classification, segmentation

CONVOLUTIONAL NEURAL NETWORKS

- A *convolutional neural network*, or CNN for short, is a type of classifier, which excels at image analysis
- CNNs can be used for many different computer vision tasks, such as image processing, classification, segmentation, and object detection.
- CNNs consist of different types of neural layers and operations:
  - **Input Layer:** Represents the input image into the CNN
  - **Convolutional Layers:** Contain the learned kernels (weights), which extract features that distinguish different images from one another
  - **Activation Function:** Applies much-needed non-linearity into the model. Non-linearity is necessary to produce non-linear decision boundaries
  - **Pooling Layers:** Gradually decreases the spatial extent of the network, which reduces the parameters and overall computation of the network
  - **Flatten Layer:** Converts a three-dimensional layer into a one-dimensional fully-connected layer for classification
  - **Output Layer:** Represents the class probabilities given the current inputs

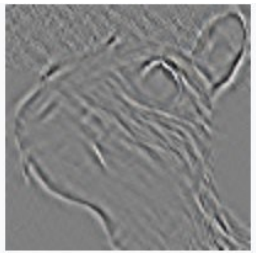
BYU

CONVOLUTIONS IN IMAGE PROCESSING

- In image processing, convolving a filter with the original image results in a range of effects, depending on the filter

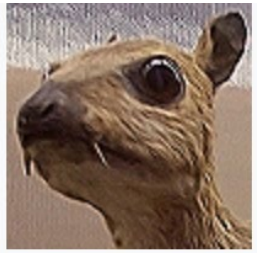
Edge detection

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Blur

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

BYU

CONVOLUTION WITH ONE FILTER

This is learned by the model

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0				

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0			

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1		

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1				

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1			

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2		

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1				

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0			

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0		

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$\circledast$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1				

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4			

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2		

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0				

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0			

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1		

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	

Feature Map

BYU

CONVOLUTION WITH ONE FILTER

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image

$*$

0	0	1
1	0	0
0	1	1

Feature Detector/
Filter

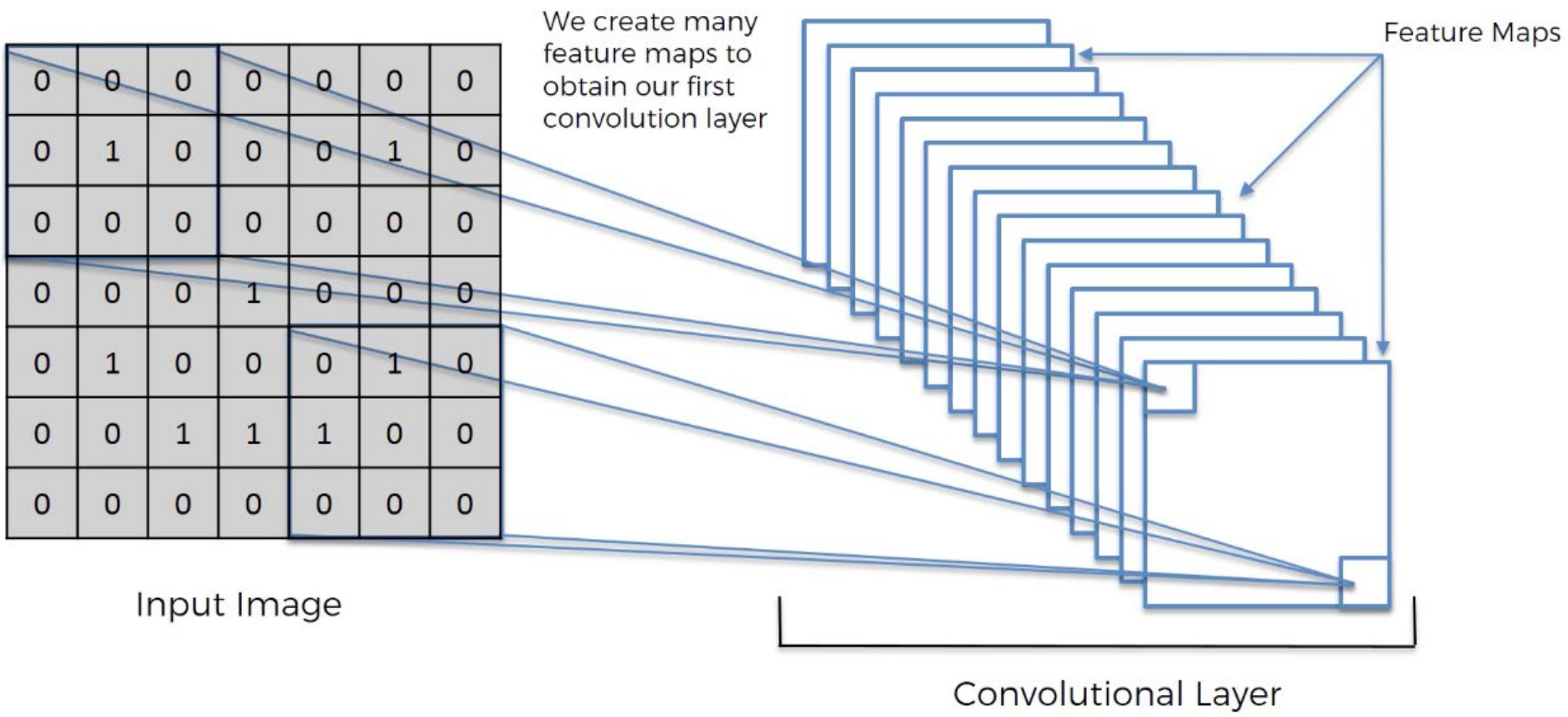
$=$

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

BYU

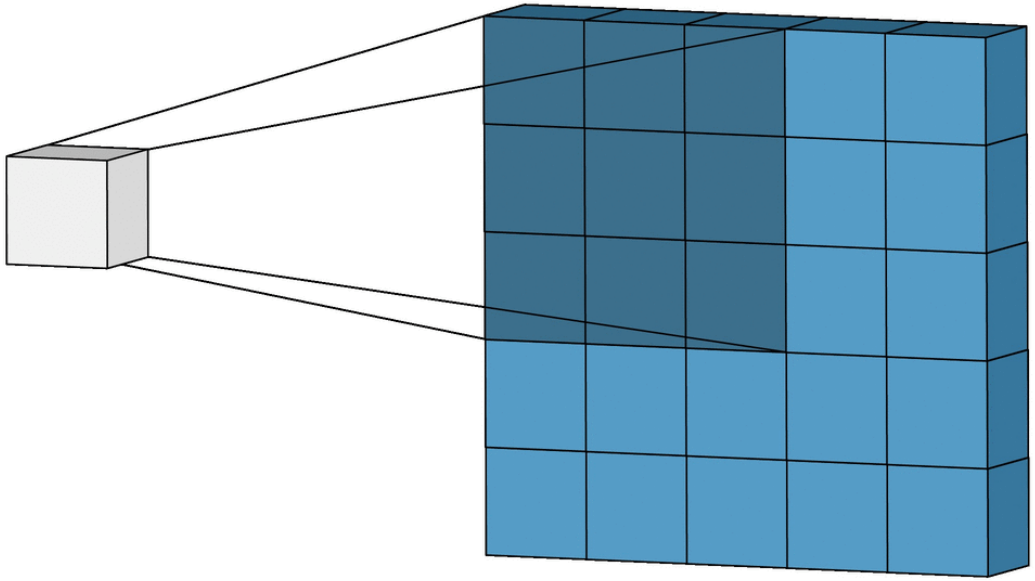
CONVOLUTIONAL LAYER



BYU

CONVOLUTION OPERATION

- During the forward pass, the kernel slides across the height and width of the image-producing the image representation of that receptive region.
- This produces a two-dimensional representation of the image known as an activation map that gives the response of the kernel at each spatial position of the image.
- The sliding size of the kernel is called a stride.



MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1		

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1	2	

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1	2	1

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1	2	1
4		

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1	2	1
4	2	

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1	2	1
4	2	1

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1	2	1
4	2	1
0		

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

1	2	1
4	2	1
0	2	

Pooled Feature Map

BYU

MAX POOLING

0	0	1	0	1
1	1	2	0	0
1	0	0	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling

Box size: 2x2
Stride = 2

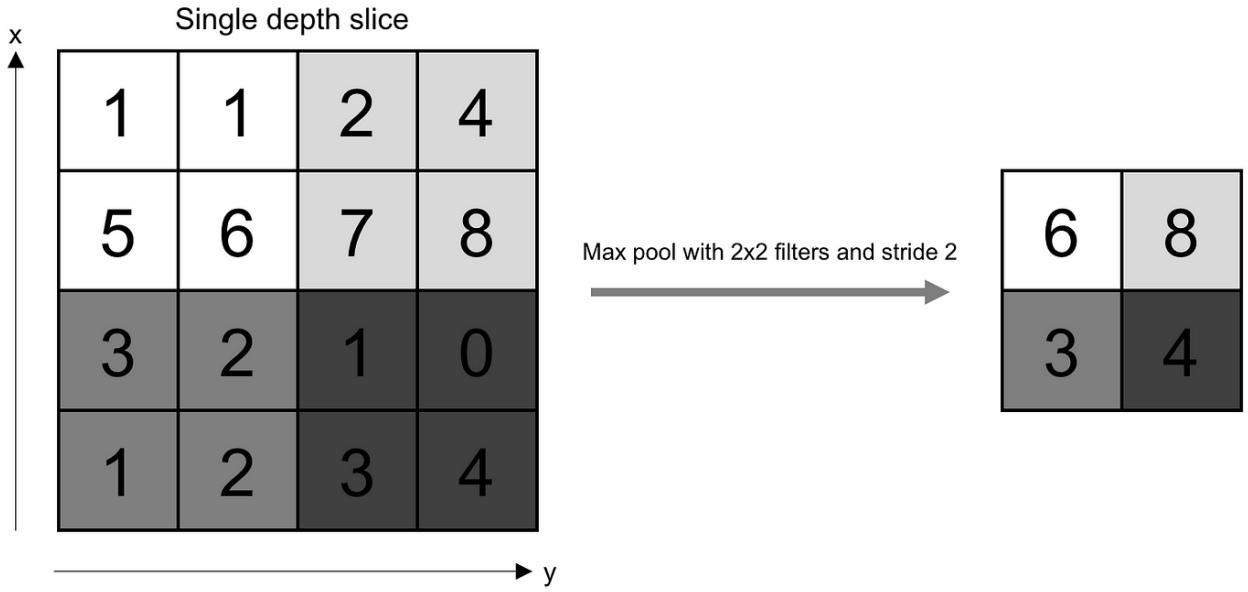
1	2	1
4	2	1
0	2	1

Pooled Feature Map

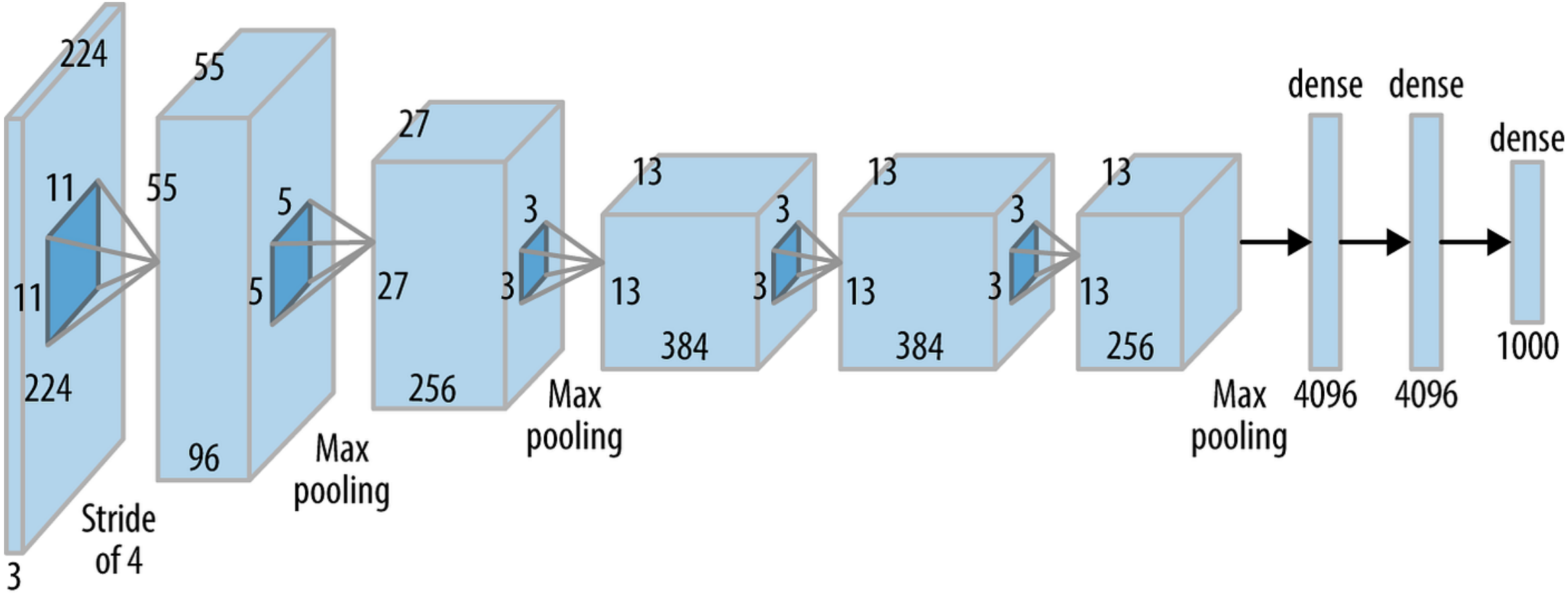
BYU

POOLING OPERATION

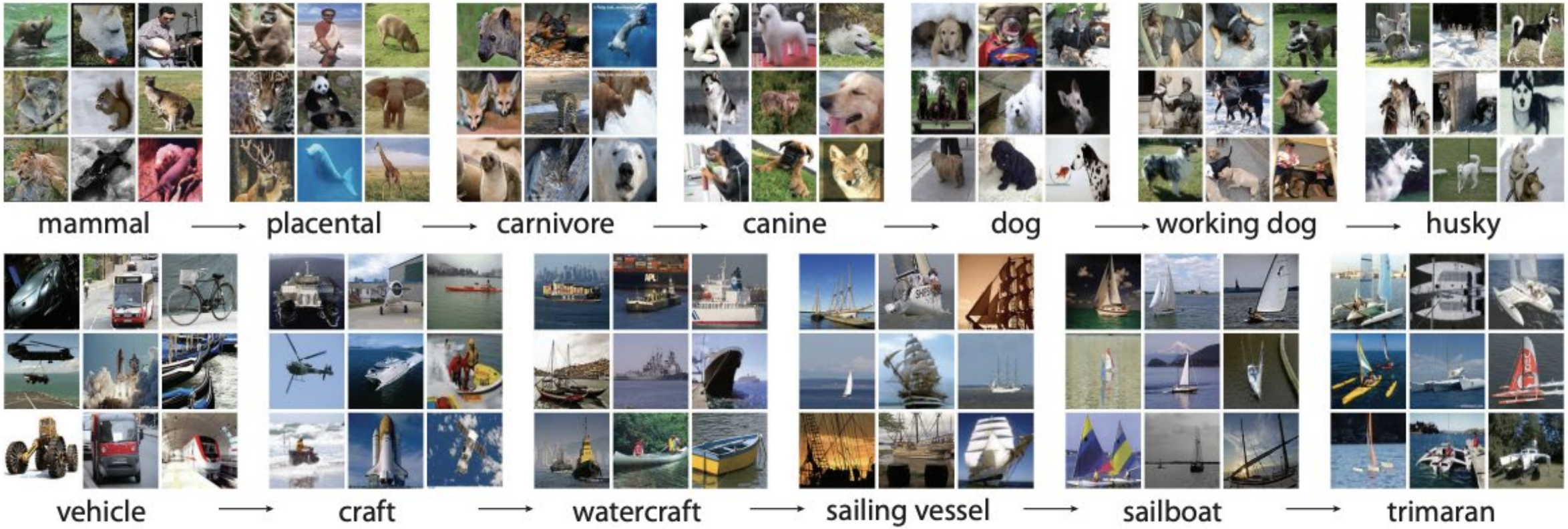
- The pooling layer replaces the output of the network at certain locations by deriving a summary statistic of the nearby outputs.
- This helps in reducing the spatial size of the representation, which decreases the required amount of computation and weights.



AlexNet (2012): PIONEERING DEEP CONVOLUTIONAL NETWORKS



IMAGENET



- The database contains over **14 million labeled images**.
- A subset of ImageNet is used for the annual ILSVRC, a competitive benchmark for image classification, object detection, and localization tasks.
- ImageNet was instrumental in the success of deep learning. In particular, the 2012 ILSVRC was a turning point when the **AlexNet model** (a deep CNN) dramatically outperformed traditional machine learning methods, showcasing the power of neural networks.

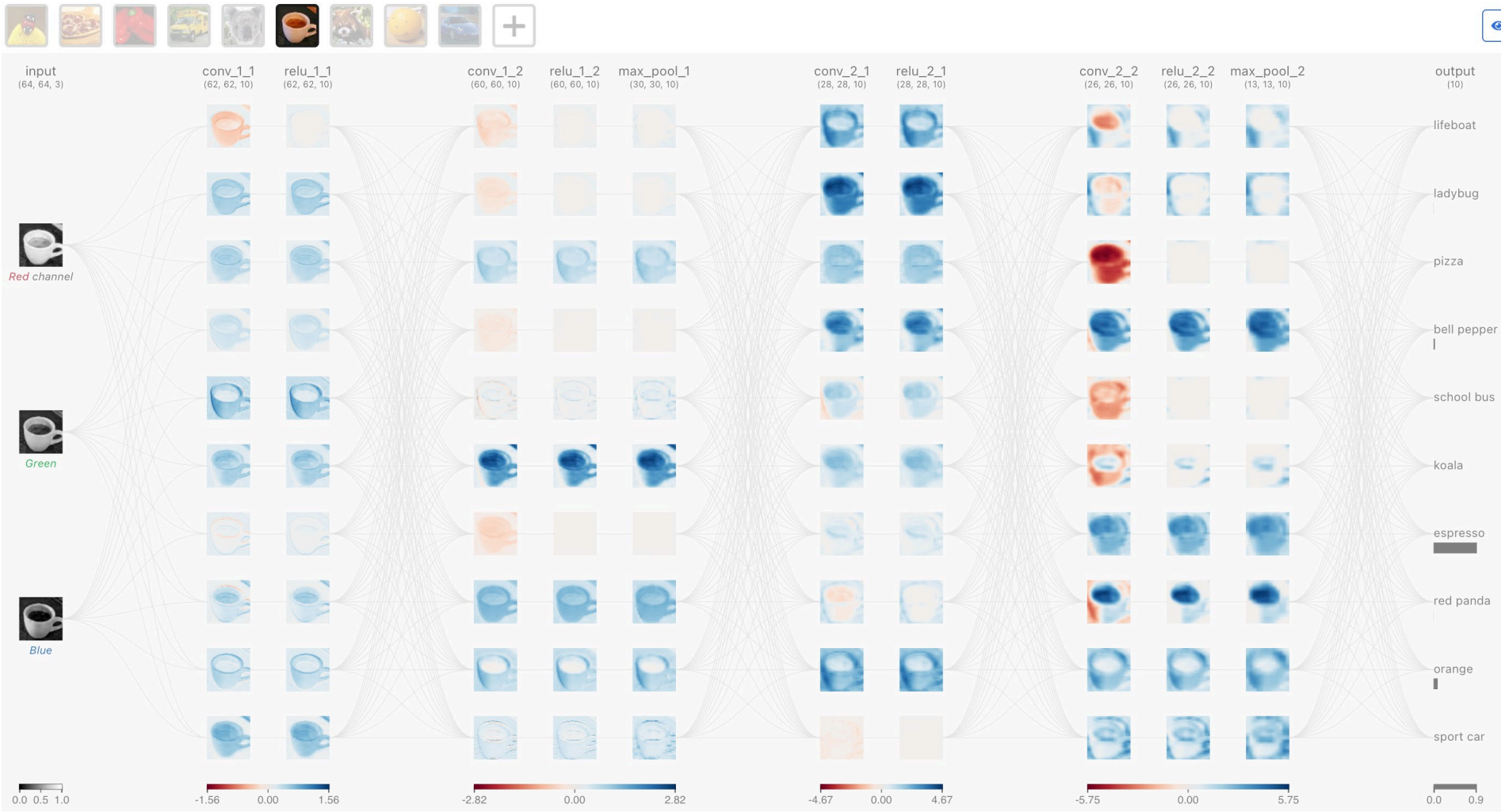
THE RACE FOR IMAGE RECOGNITION SUPREMACY

Network	Year	Salient Feature	Top-5 Accuracy	Params	FLOPs
AlexNet	2012	First deep CNN breakthrough	84.7%	62M	1.5B
VGGNet	2014	Deep with fixed 3×3 kernels	92.3%	138M	19.6B
Inception (GoogLeNet)	2014	Parallel “wider” kernels	93.3%	6.4M	2B
ResNet-152	2015	Residual/skip connections	95.5%	60.3M	11B
DenseNet-201	2016	Dense connectivity (feature reuse)	95.8%	20M	~4B
MobileNetV2	2018	Depthwise separable convs	~90%	3.4M	300M
EfficientNet-B7	2019	Compound scaling (depth/width/res)	97.1%	66M	37B
ConvNeXt-L	2022	ConvNet redesigned for transformer-level performance	~87% top-1	198M	~4B

- **AlexNet vs. ResNet-152:** Both have ~60M parameters, but ResNet-152 delivers **~10% higher top-5 accuracy** and needs **much more computation**, increasing training cost.
- **VGGNet:** Far **more parameters and FLOPs** than ResNet-152 but with **lower accuracy**, making it slower and less efficient.
- **Training Cost:** AlexNet and ResNet-152 train in **similar time**, but ResNet-152 uses **~10× less memory** and achieves **~9% better accuracy**.
- **EfficientNet:** Uses **compound scaling** to achieve **state-of-the-art accuracy with fewer FLOPs**—much more efficient than VGG/ResNet fsor similar performance.
- **ConvNeXt:** A modernized ConvNet offering **transformer-level accuracy** while keeping **CNN-style efficiency** and simpler training.

BYU

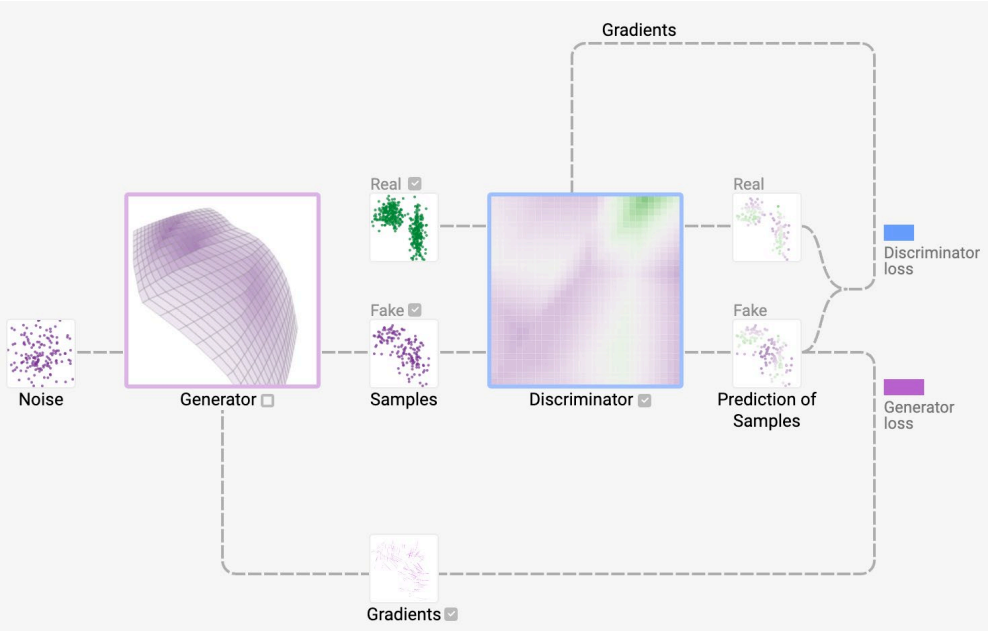
CNN EXPLAINER: INTERACTIVE EXPERIMENTATION

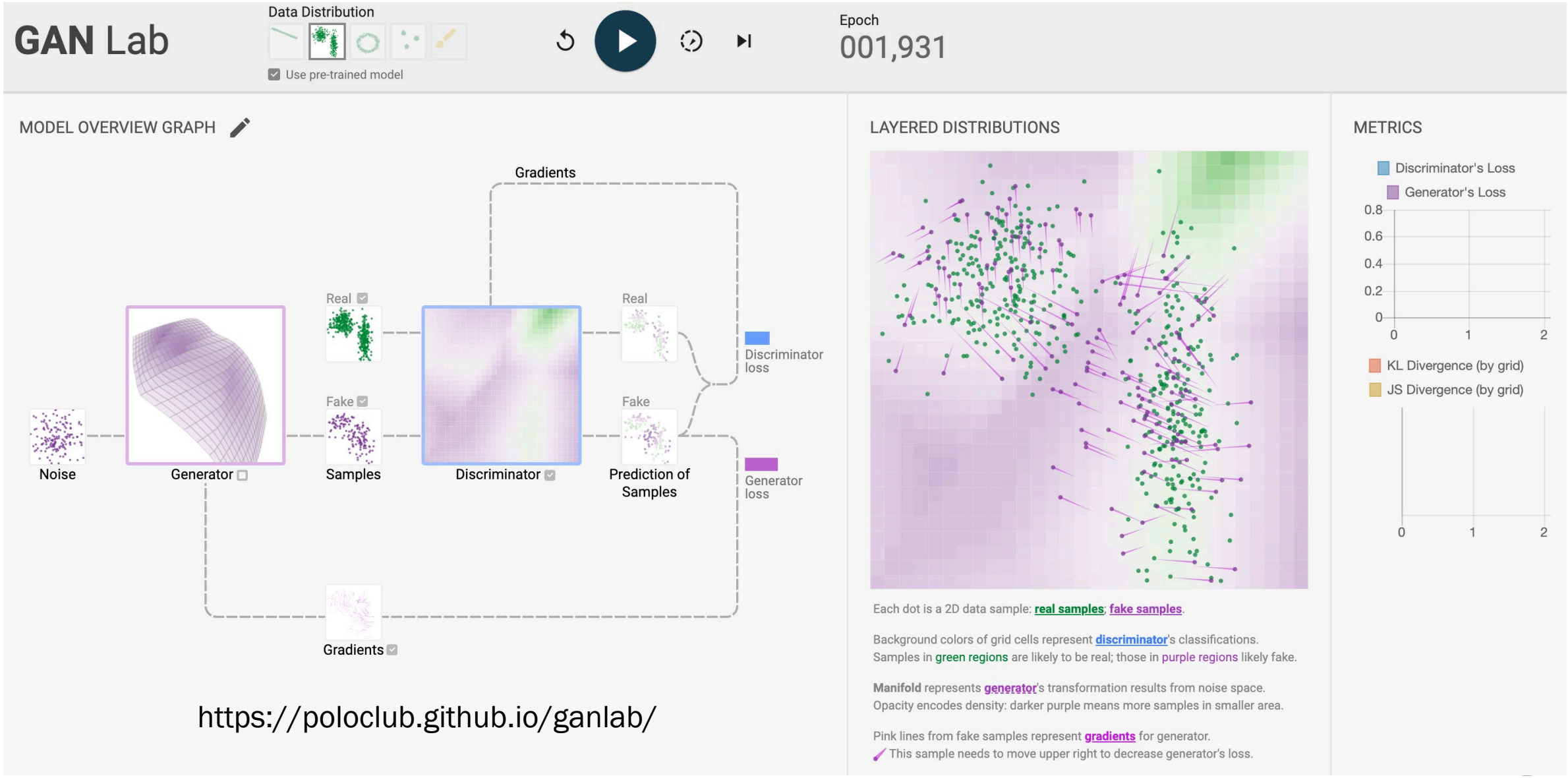


https://poloclub.github.io/cnn-explainer/

GENERATIVE ADVERSARIAL NETWORKS (GAN)

- GANs learn to generate realistic data (images, text, audio) by setting up a competition.
- They consist of two neural networks:
  - Generator: creates fake data
  - Discriminator: tries to tell real data from fake
- Training is adversarial:
  - The generator tries to fool the discriminator
  - The discriminator tries to catch the fakes
- Over time, both improve, and the generator produces data that looks real
- Intuition: Like a counterfeiter and a detective getting better by competing against each other.

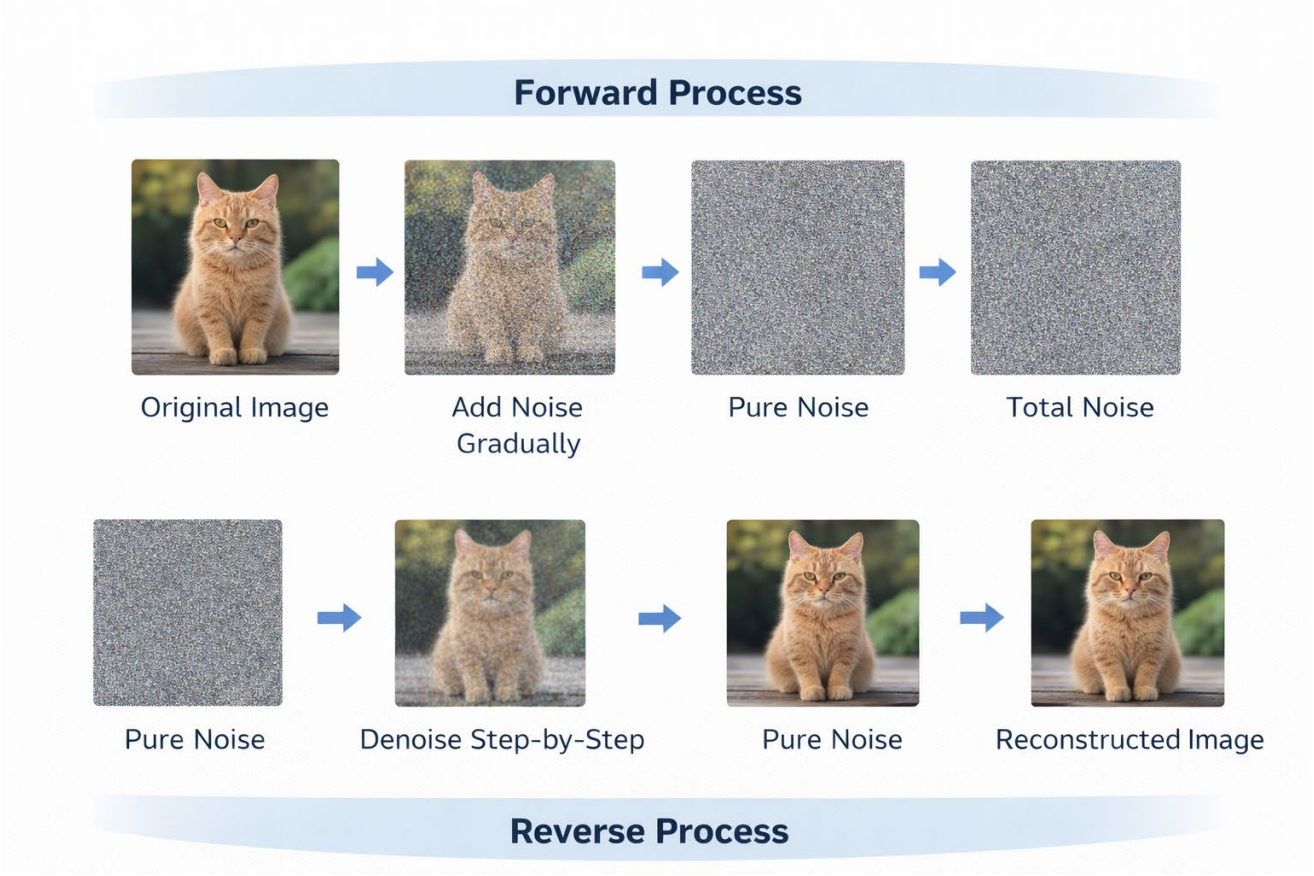




https://poloclub.github.io/ganlab/

DIFFUSION MODELS

- What are diffusion models?
  - A class of generative models used to create realistic data (images, audio, text)
- Core idea
  - Start with real data and gradually add noise
  - Learn how to reverse this process to recover the original data
- How they work (intuition)
  - Forward process: slowly corrupt data with random noise
  - Reverse process: model learns to remove noise step-by-step
- Why they matter
  - Can generate high-quality, realistic outputs
  - Power modern tools like AI image generators (e.g., DALL·E, Stable Diffusion)
- Key advantage
  - More stable and easier to train than some older generative models (like GANs)
  - Learn the structure of data by learning how to “denoise” it
  - Turn randomness into meaningful patterns

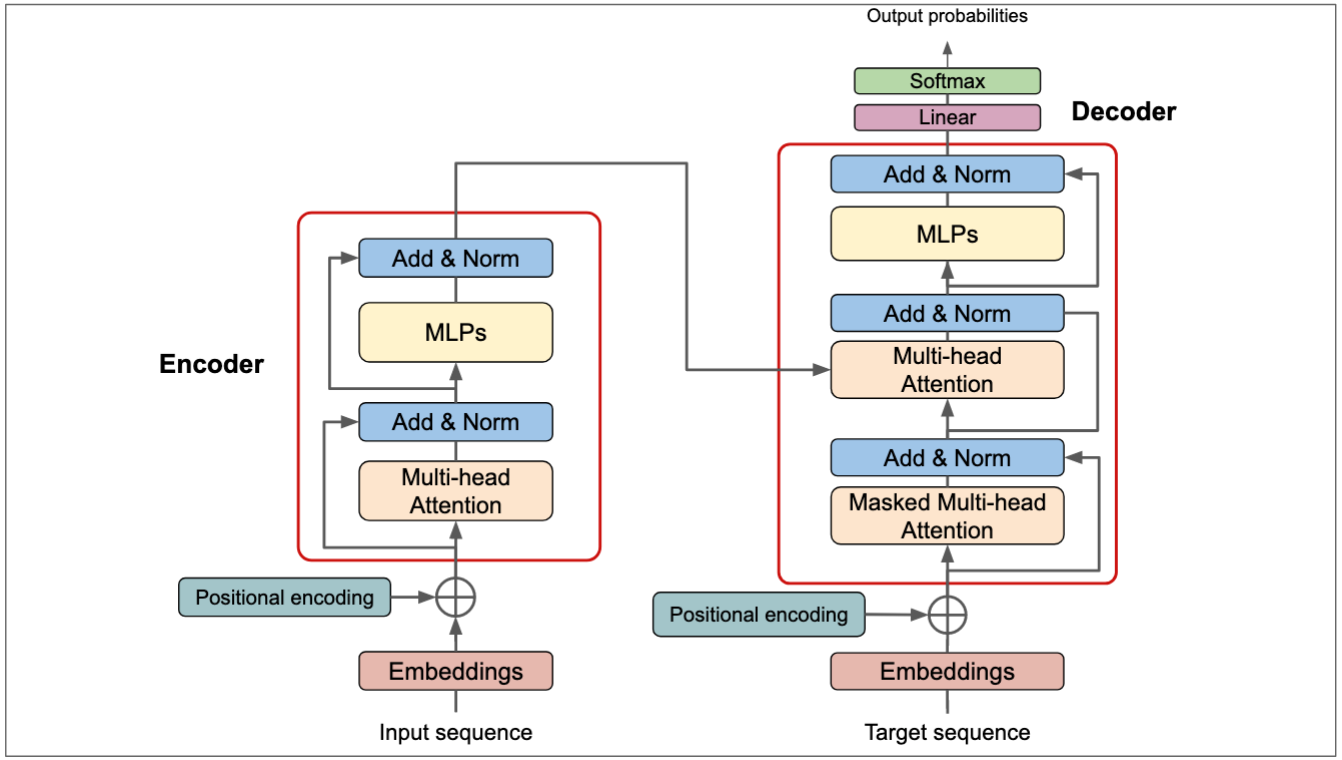


TRANSFORMER NETWORKS

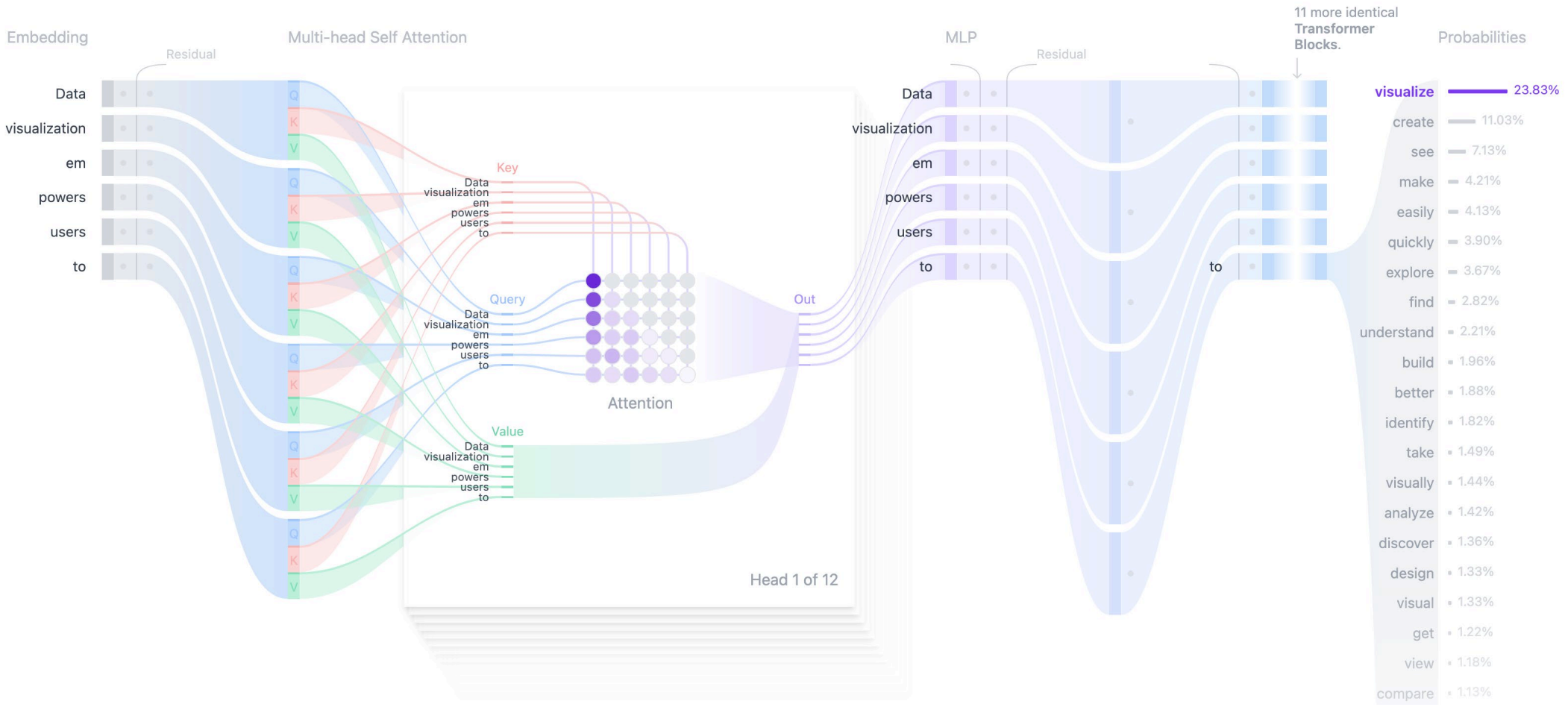
- Transformers generate text one word at a time
  - Given a prompt, the model predicts the most likely next word
- Their power comes from self-attention
  - Each word decides which words matter
  - This allows the model to capture long-range context

Three core components:

- Embeddings — words → vectors
  - Text is split into tokens
  - Each token becomes a vector that captures meaning
- Transformer Blocks — the “thinking”
  - Self-Attention: tokens focus on relevant words
  - Feed-Forward Network (MLP): refines each token after context is added
- Output Layer — prediction
  - A linear layer + softmax produces probabilities for the next word



TRANSFORMER EXPLAINER



https://poloclub.github.io/transformer-explainer/

BYU

LLM VISUALIZATION

- Large Language Models for the curious beginner (3blue1brown)
  https://www.youtube.com/watch?v=LPZh9BOjkQs&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=5

BYU

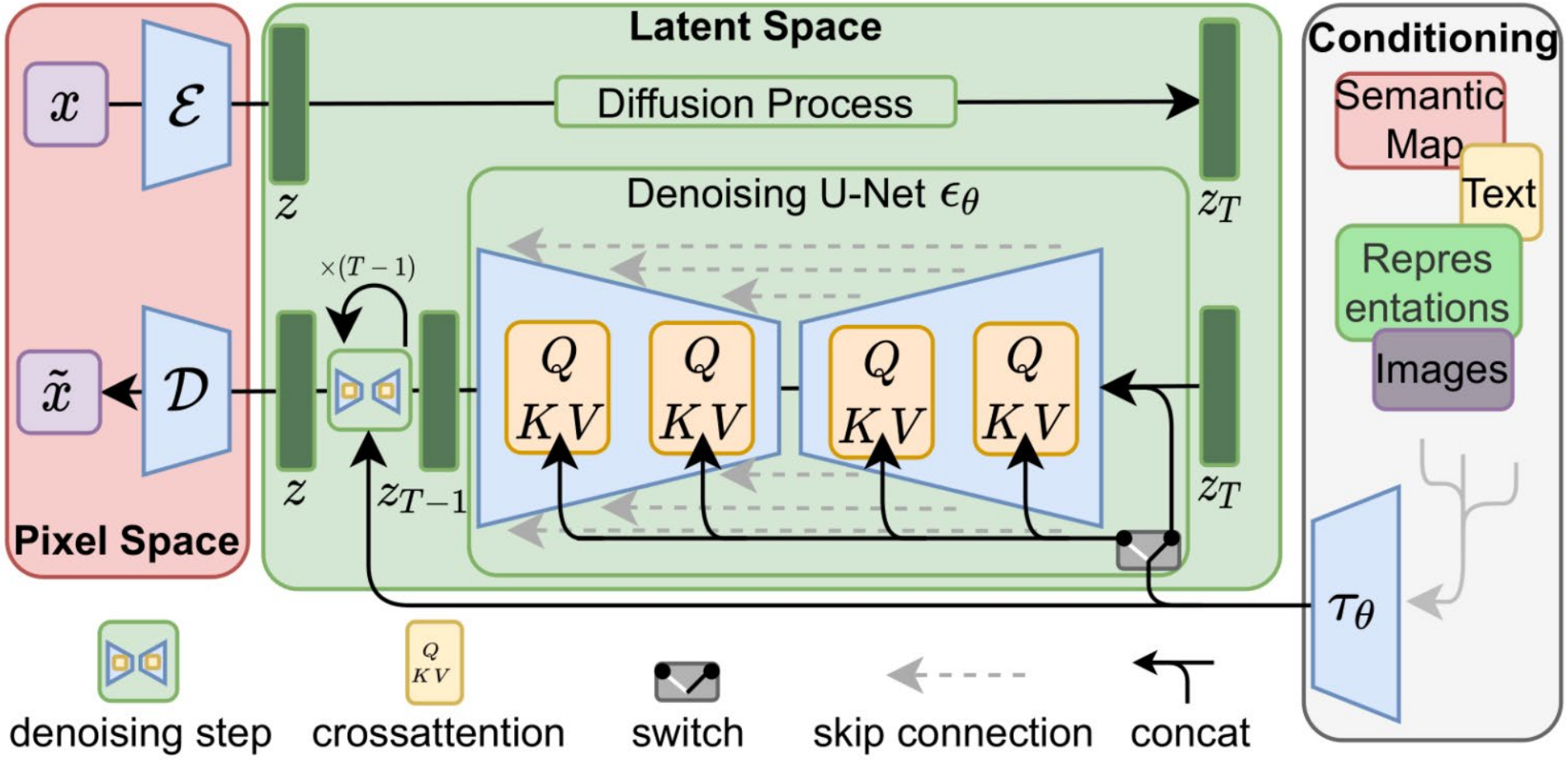
STABLE DIFFUSION NETWORKS

- Stable Diffusion is a text-to-image model that generates high-resolution images from text prompts.
- We break down the image generation process into three main steps:

1. Text Representation Generation: Stable Diffusion converts a text prompt into a text vector representation.
2. Image Representation Refining: Starting from random noise, Stable Diffusion refines the image representation gradually, guided by the text representation. Stable Diffusion repeats the refining over multiple timesteps (50 in our Diffusion Explainer).
3. Image Upscaling: Stable Diffusion upscales the image representation into a high-resolution image.

BYU

LATENT DIFFUSION



TEXT REPRESENTATION (TOKENIZATION AND ENCODING)

1. Stable Diffusion tokenizes a text prompt into a sequence of tokens. For example, it splits the text prompt "a cute and adorable bunny" into the tokens a, cute, and, adorable, and bunny.
2. Also, to mark the beginning and end of the prompt, Stable Diffusion adds <start> and <end> tokens at the beginning and the end of the tokens. The resulting token sequence for the above example would be <start>, a, cute, and, adorable, bunny, and <end>.

- **Text Encoding**: To use the text representation for guiding image generation, Stable Diffusion ensures that the text representation contains the information related to the image depicted in the prompt. This is done by using a special neural network called CLIP.
- CLIP, which consists of an **image encoder and a text encoder**, is trained to encode an image and its text description into vectors that are similar to each other. Therefore, the text representation for a prompt computed by CLIP's text encoder is likely to contain information about the images described in the prompt.

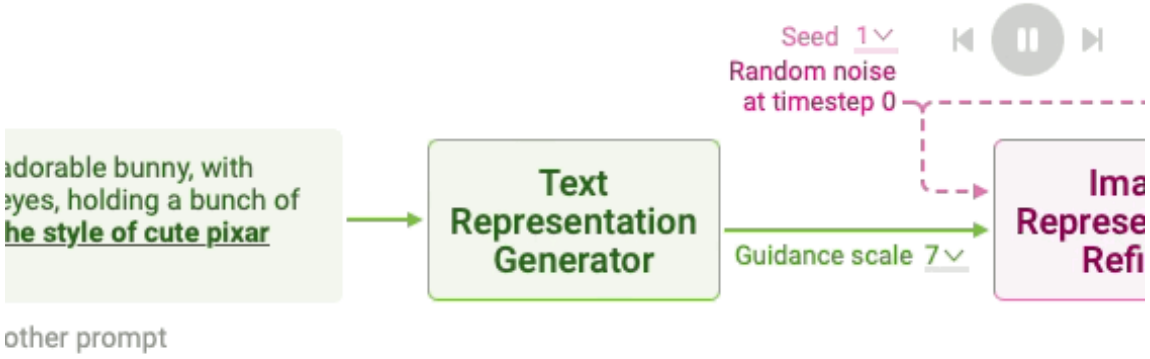
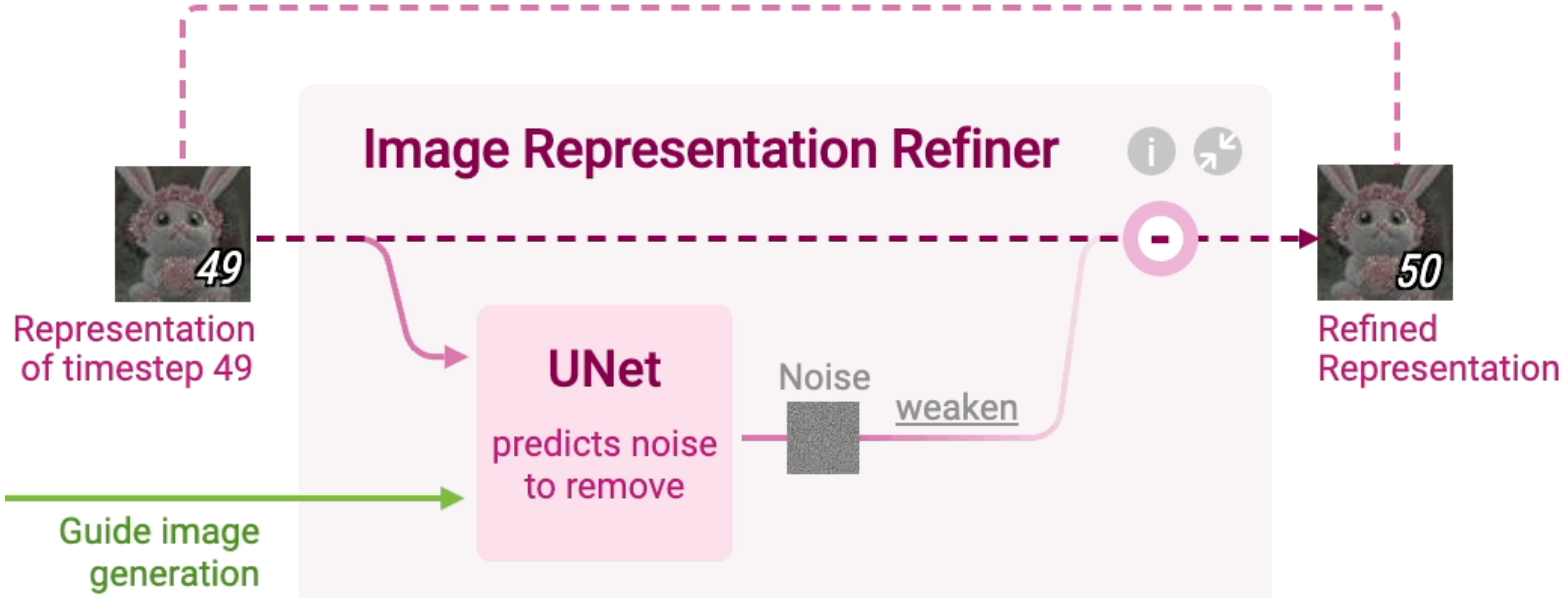
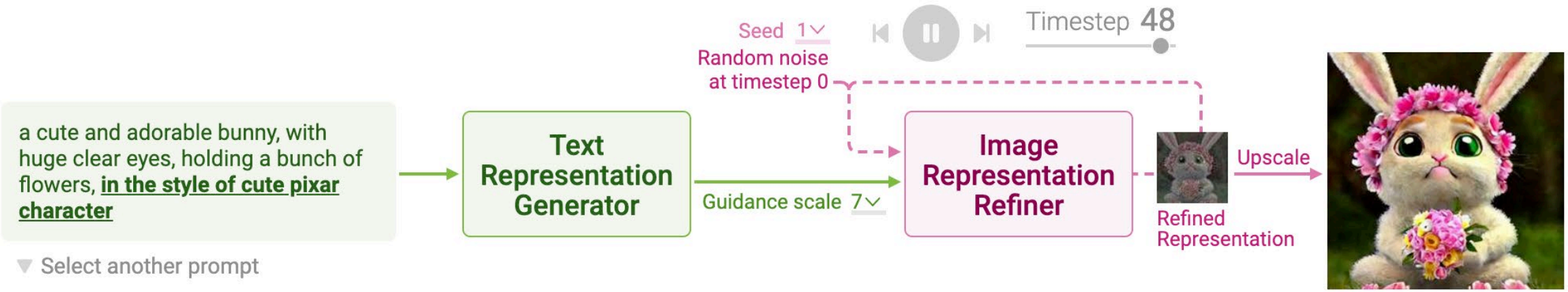


IMAGE REFINER

- Stable Diffusion generates a vector that numerically summarizes a high-resolution image depicted in the text prompt.
- This is done by refining a randomly initialized noise over multiple timesteps to gradually improve the image quality and adherence to the prompt
- At each timestep, a neural network called UNet predicts noise in the image representation of the current timestep.
- UNet takes three inputs:
  1. Image representation of the current timestep
  2. Text representation of the prompt to guide what noise should be removed from the current image representation to generate an image adhering to the text prompt
  3. Timestep to indicate the amount of noise remaining in the current image representation



STABLE DIFFUSION EXPLAINER



https://poloclub.github.io/diffusion-explainer/

REFERENCES

- 3blue1brown Transformer Networks: https://www.youtube.com/watch?v=wjZofJX0v4M&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=6
- https://medium.com/@prathammodi001/convolutional-neural-networks-for-dummies-a-step-by-step-cnn-tutorial-e68f464d608f
- Convolutional Neural Networks: A Comprehensive Guide: https://medium.com/thedeephub/convolutional-neural-networks-a-comprehensive-guide-5cc0b5eae175

BYU